Locator handling for TextFieldElement:

---

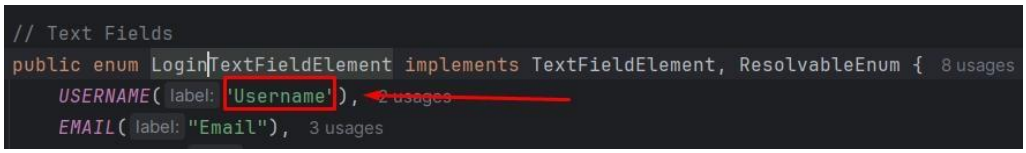
1 Locator Template (properties file)

- **Description:** A raw XPath template with %s as a placeholder.
- **Purpose:** One template → reusable for multiple fields. Keeps locators DRY and easy to maintain.
- **OOP Principle: Abstraction — hides locator complexity behind a simple template.**

---

2 Enum Field Definition

- **Description:** Enum constants represent UI fields, each storing a label.
- **Purpose:** Strong typing + IDE autocomplete. Labels double as arguments for templates.
- **OOP Principle: Encapsulation — field definitions + labels are bundled together.**



---

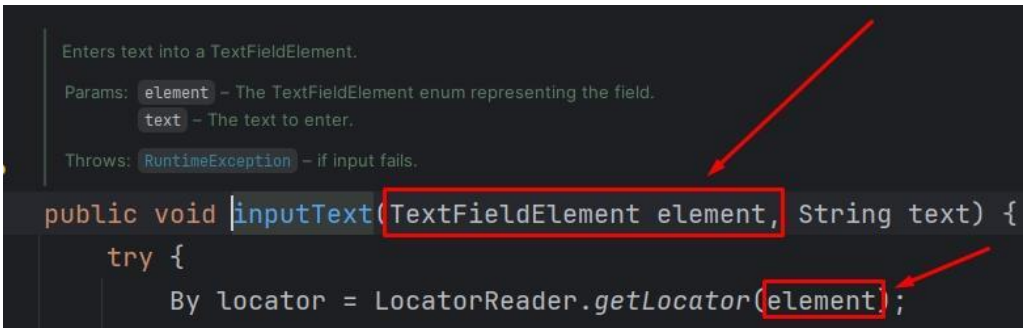
3 Enum Arguments via getArgs()

- **Description:** Enum exposes its label as an argument.
- **Purpose:** Clean injection of field names into locator templates. Flexible resolution.
- **OOP Principle: Polymorphism — every enum can expose args differently if needed.**

---

4 LocatorReader (Resolver)

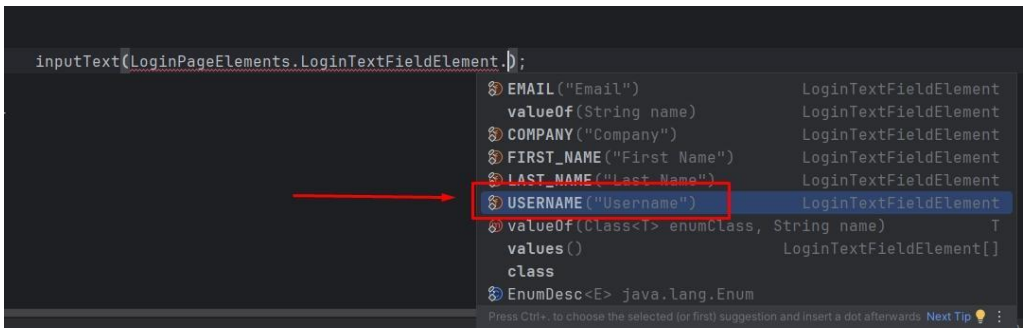
- **Description:** Reads templates, fetches args, applies them → final XPath.
- **Purpose:** Decouples locators from code, ensures consistent resolution.
- **OOP Principle: Abstraction + Encapsulation — resolution logic is hidden and reusable.**

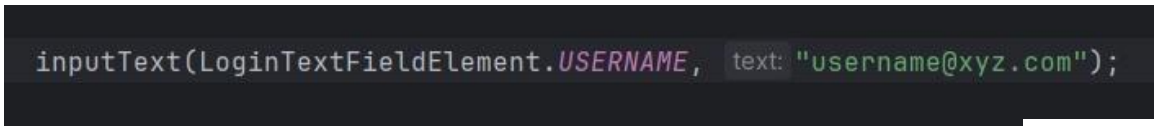


---

5 Clean Test Call

- **Description:** The code you actually write in tests.
- **Purpose:** Readable, no raw XPath — framework handles plumbing.
- **OOP Principle: Inheritance** — test code benefits from reusable parent logic (framework).

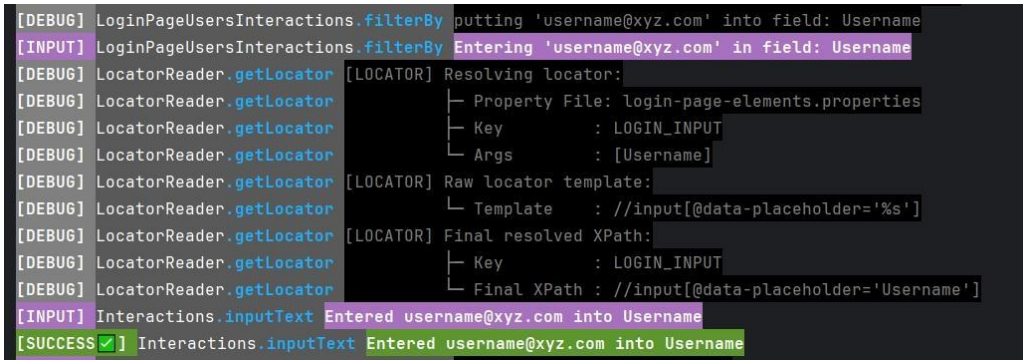




---

6 Runtime Logs (End-to-End)

- **Description:** Transparent trace of what happened.
- **Purpose:** Debugging made easy — file, key, args, resolved XPath, and action result all shown.
- **OOP Principle: Encapsulation** — internals are visible in logs only, not mixed into test code.



✨ **TL;DR:**

- **Abstraction** → Templates hide complexity.
- **Encapsulation** → Enums bundle field + label.
- **Polymorphism** → Flexible `getArgs()` handling.
- **Inheritance** → Tests stay clean, extend framework behavior.
